

UNIT-1

- Prof. Prachi Jain

Course Title: DevOpS: SOFTWARE DEVELOPMENT & IT OPERATIONS

Semester	VII	Teaching Scheme				Evaluation Scheme				
						Theory			Practical	
Term	ODD	Th	Tu	Pr	Credits	TAE	CAE	ESE	INT	EXT
Course Category	PEC4	4			4	10	15	50	-	-
Course Code	UNITT408									
Teaching Mode	Offline	4 Hrs			Total	75				
Duration of ESE	2:30 Hrs.					75				

Unit - 1

▶ **INTRODUCTION TO DEVOPS**

▶ Concepts of

- ▶ SDLC

- ▶ Agile

- ▶ ITIL

▶ Need for DevOps

▶ History of DevOps

▶ Role of a DevOps Engineer

▶ Terminologies in DevOps

▶ JIRA

What is DevOps?

- ▶ DevOps is a culture which promotes collaboration between Development and Operations Team to deploy code to production faster in an automated & repeatable way.
- ▶ The word 'DevOps' is a combination of two words 'development' and 'operations.'

DevOps Principles

- ▶ The main principles of DevOps are automation, continuous delivery, and fast reaction to feedback.
- ▶ DevOps pillars in the **CAMS** acronym:
 - ▶ Culture represented by human communication, technical processes, and tools
 - ▶ Automation of processes
 - ▶ Measurement of KPIs (Key Performance Indicators)
 - ▶ Sharing feedback, best practices, and knowledge

DevOps

- ▶ DevOps is a modern approach to software development that **brings development and operations teams together to deliver applications faster and more reliably.**
- ▶ It focuses on collaboration, automation, and continuous improvement across the software lifecycle.
- ▶ DevOps is like a teamwork approach for making and delivering computer programs.
 - ▶ Encourages close collaboration between development and operations teams.
 - ▶ Automates build, test, and deployment processes to reduce errors.
 - ▶ Enables faster and more frequent software releases.
 - ▶ Improves system reliability, monitoring, and feedback loops.

15 MOST POPULAR DEVOPS TOOLS TO LEARN IN 2026

TOP DEVOPS TOOLS 2026: A VISUAL OVERVIEW

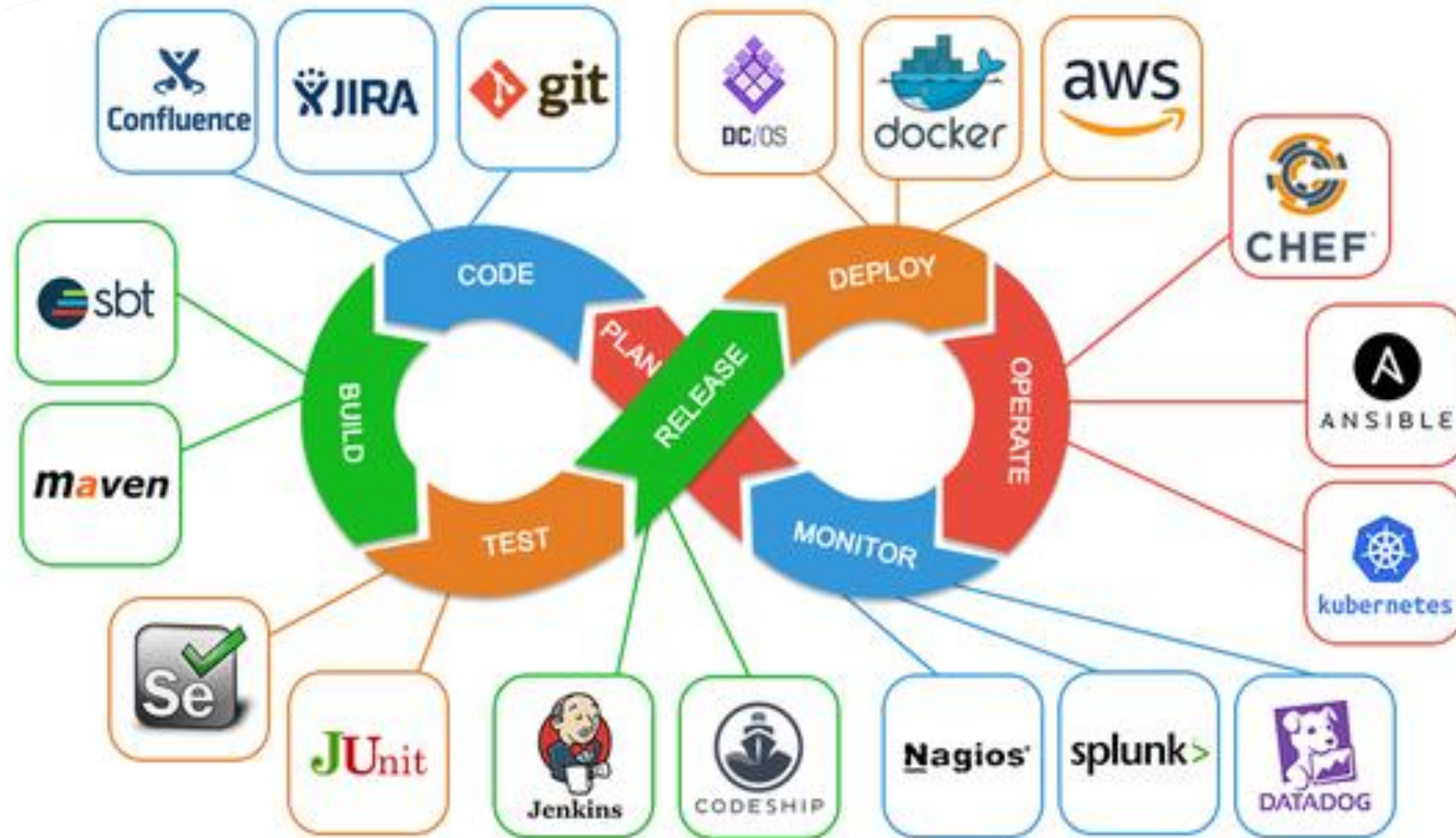


7 Steps to Successful DevOps Adoption



- ▶ Containerization is a software deployment method that packages an application's code, libraries, and dependencies into a single lightweight executable called a container.
- ▶ This ensures the application runs consistently and uniformly across any computing environment—from a developer's laptop to a massive cloud production server.

DevOps Lifecycle



DevOps Lifecycle

- ▶ The DevOps Lifecycle is a continuous, looping process that bridges the gap between software development (Dev) and IT operations (Ops).

- ▶ **Phases of DevOps Lifecycle**

Phase 1: Plan (Dev)

Phase 2: Code (Dev)

Phase 3: Build (Dev)

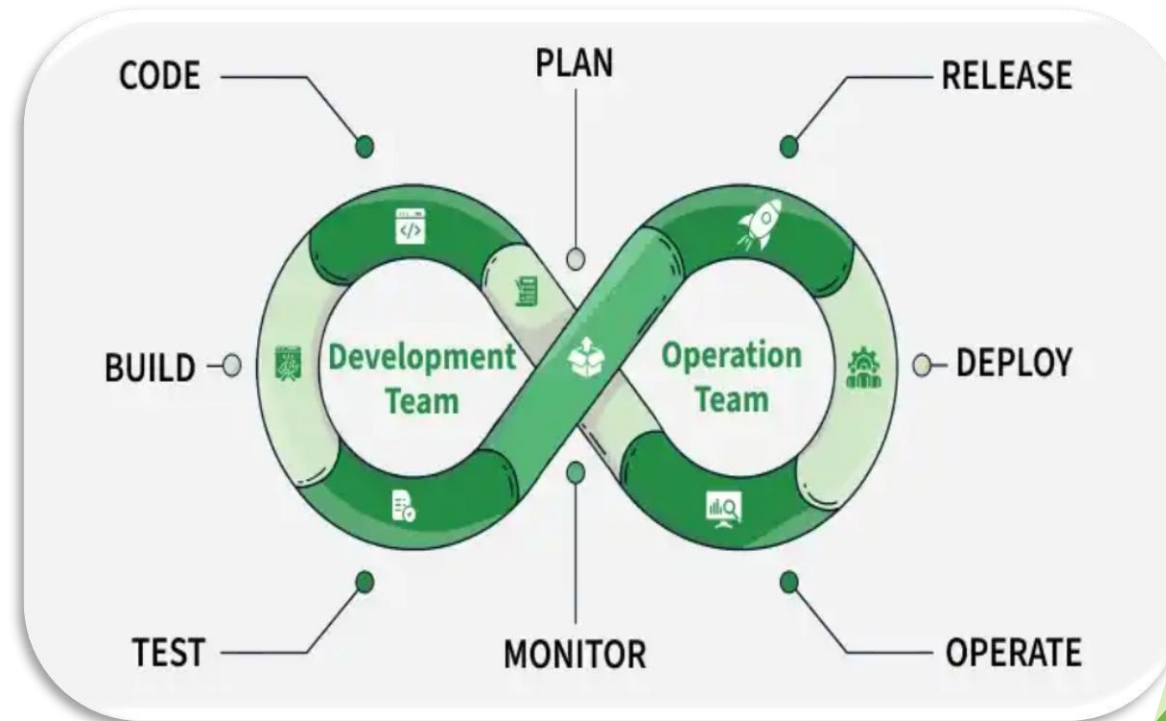
Phase 4: Test (Dev)

Phase 5: Release (Ops)

Phase 6: Deploy (Ops)

Phase 7: Operate (Ops)

Phase 8: Monitor (Ops)



Phase 1: Plan (Dev)

Before a single line of code is written, the team must define the value they are creating.

Goal: Define requirements, create roadmaps, and organize tasks.

Key Activities: Backlog management, sprint planning, user story creation.

Tools: Jira, Trello, Azure Boards.

Phase 2: Code (Dev)

Developers write the code and commit it to a shared repository.

Goal: Write high-quality, version-controlled code.

Key Activities: Coding, code reviews, merging branches.

Tools: Git, GitHub, GitLab, Bitbucket.

Phase 3: Build (Dev)

Once code is committed, it is pulled and compiled into an executable format.

Goal: Detect compilation errors early and package the application.

Key Activities: Compiling code, dependency management, creating container images (Docker).

Tools: Maven, Gradle, Docker, Packer.

Phase 4: Test (Dev)

- ▶ The build is tested for bugs, security vulnerabilities, and performance issues.
- ▶ **Goal:** Ensure software quality and prevent bugs from reaching production.
- ▶ **Key Activities:** Unit testing, integration testing, static code analysis.
- ▶ **Tools:** Selenium, JUnit, SonarQube, TestNG.

Phase 5: Release (Ops)

- ▶ The code has passed testing and is ready to be deployed. This phase manages the versioning and approval.
- ▶ **Goal:** Schedule and manage the release to specific environments (Staging/Prod).
- ▶ **Key Activities:** Artifact versioning, change management approvals.
- ▶ **Tools:** Jenkins, GitLab CI, CircleCI.

Phase 6: Deploy (Ops)

- ▶ The application is pushed to the production servers.
- ▶ **Goal:** Deliver the software to end-users with zero downtime.
- ▶ **Key Activities:** Infrastructure provisioning (IaC), configuration management, blue/green deployments.
- ▶ **Tools:** Terraform, Ansible, Kubernetes (ArgoCD), AWS CodeDeploy.

Phase 7: Operate (Ops)

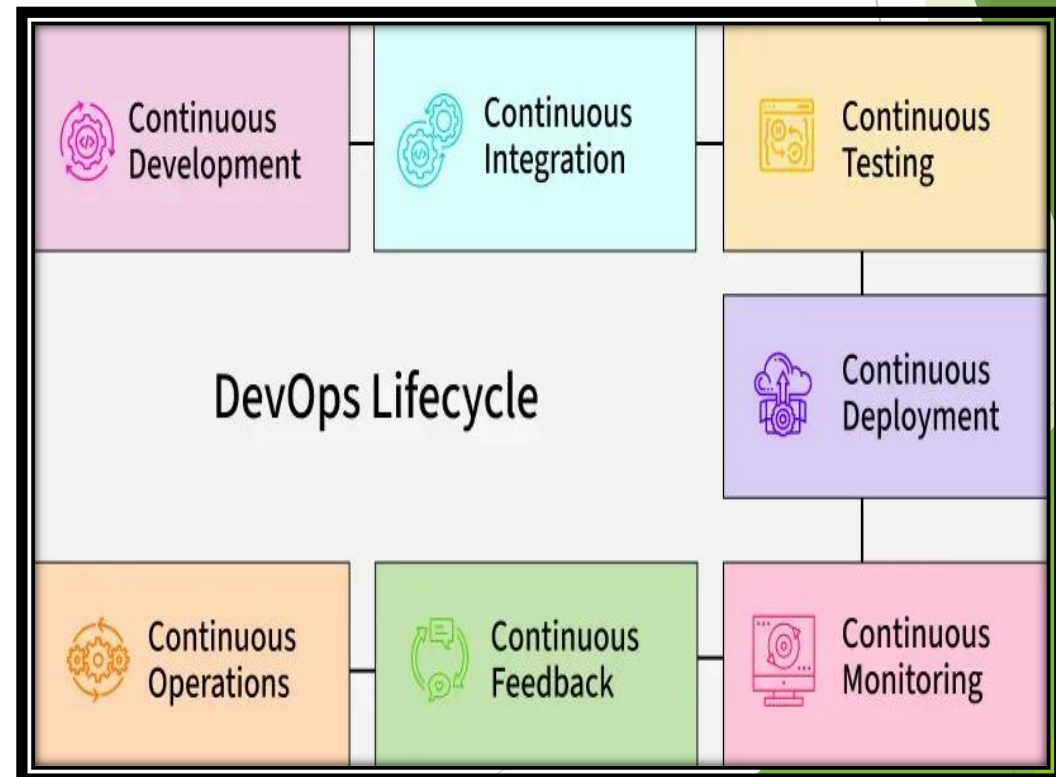
- ▶ The software is live. This phase involves the day-to-day management of the infrastructure running the app.
- ▶ **Goal:** Ensure system uptime and reliability.
- ▶ **Key Activities:** Server patching, auto-scaling, database management.
- ▶ **Tools:** Chef, Puppet, PowerShell, Ansible.

Phase 8: Monitor (Ops)

- ▶ Data is collected from the live application to understand user behavior and system performance. This feedback loops back into the **Plan** phase.
- ▶ **Goal:** Detect issues immediately and gather insights for future improvements.
- ▶ **Key Activities:** Log analysis, performance tracking, alerting.
- ▶ **Tools:** Prometheus, Grafana, Nagios, ELK Stack (Elasticsearch, Logstash, Kibana).

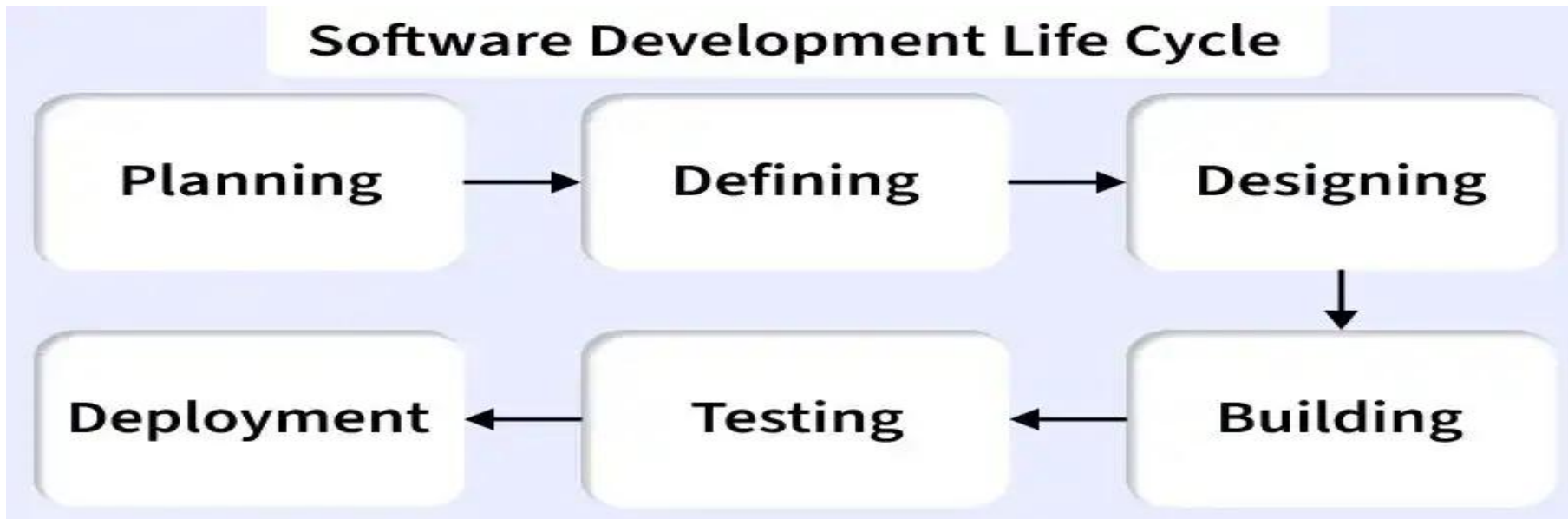
7 Cs of DevOps

- ▶ **7 Cs of DevOps**
- ▶ The 7 Cs of DevOps are core principles that help make DevOps successful.
- ▶ They guide how teams work together, build, test, and deliver software faster and more reliably. Each of these Cs contributes to a workflow that enhances the quality, speed, and reliability of delivering software products:
- ▶ Continuous Development
- ▶ Continuous Integration
- ▶ Continuous Testing
- ▶ Continuous Deployment/Continuous Delivery
- ▶ Continuous Monitoring
- ▶ Continuous Feedback
- ▶ Continuous Operations



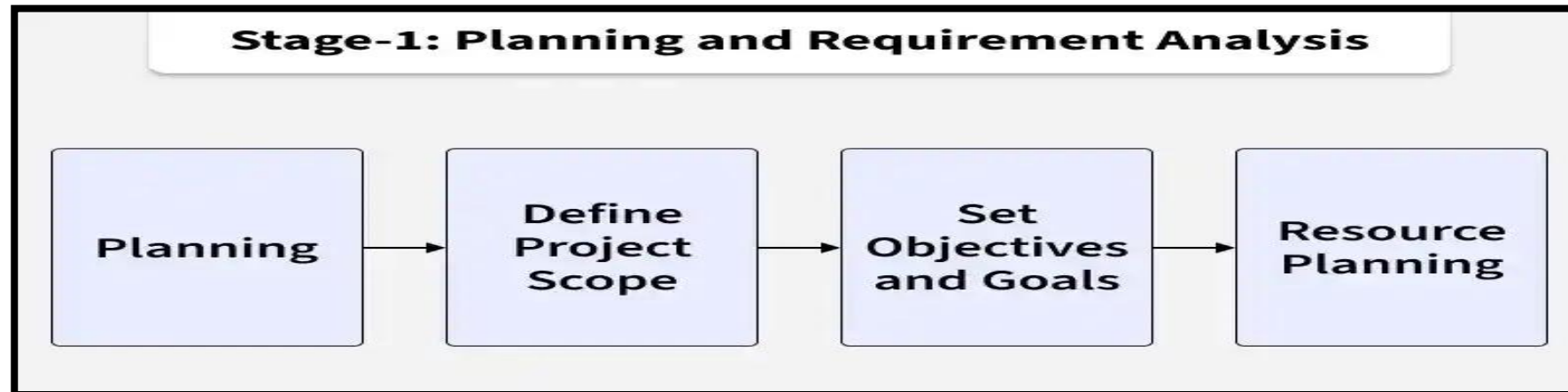
Software Development Life Cycle (SDLC)

- ▶ The Software Development Life Cycle (SDLC) is a structured process used to plan, design, develop, test, deploy, and maintain software. It ensures a systematic workflow and helps align software development with business goals and user requirements.
- ▶ Provides a clear and organized framework for managing development phases
- ▶ Helps in early detection of defects, reducing overall cost and time
- ▶ Ensures high-quality software delivery that meets user expectations

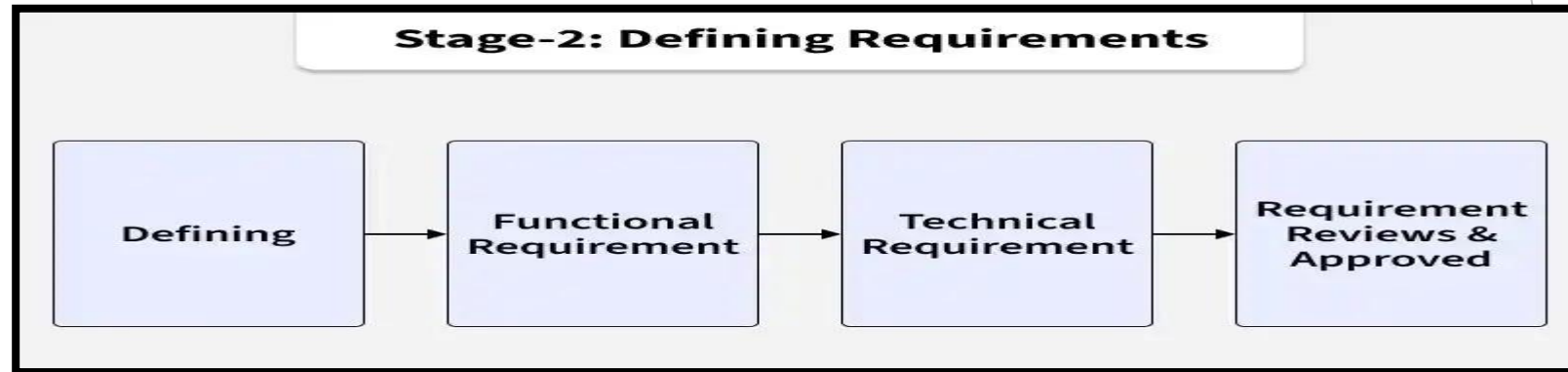


Stages of the Software Development Life Cycle

- ▶ The Software Development Life Cycle (SDLC) typically consists of six or seven stages, depending on the development model used.
- ▶ **Stage 1: Planning & Feasibility Analysis**
- ▶ This stage determines whether the project is technically, financially, and operationally feasible.
- ▶ **Activities:** Feasibility analysis, cost estimation, scheduling, resource planning
- ▶ **Output:** Project Plan, Feasibility Report
- ▶ **Key Roles:** Project Managers, Senior Engineers, Stakeholders



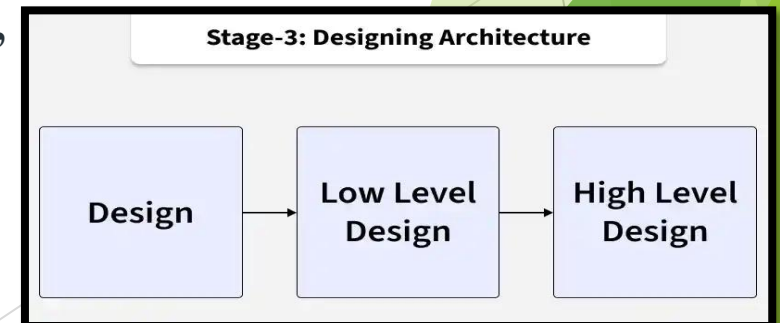
- ▶ **Stage 2: Requirement Specification (SRS)**
- ▶ In this stage, detailed functional and non-functional requirements are documented clearly and approved by stakeholders.
- ▶ **Activities:** Requirement gathering, validation, documentation
- ▶ **Output:** Software Requirement Specification (SRS)
- ▶ **Key Roles:** Business Analysts, Product Owners



Stage 3: System Design

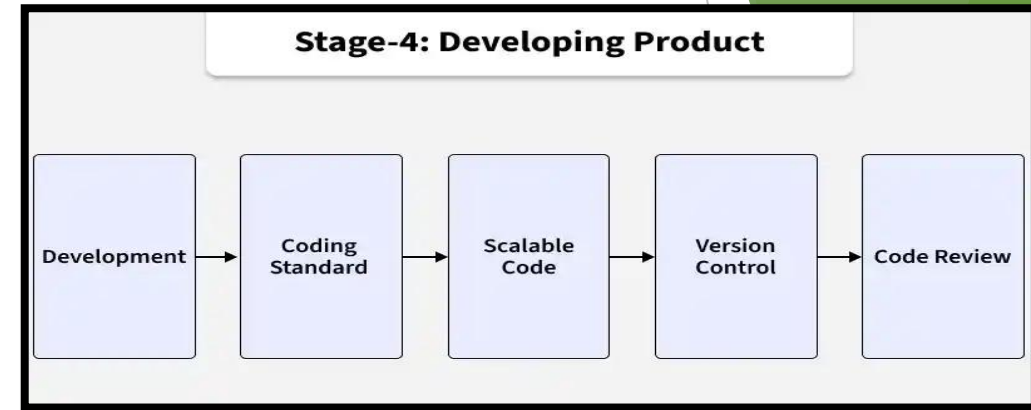
In this stage, the approved requirements are transformed into a technical blueprint for implementation.

- **High-Level Design (HLD):** Defines system architecture, technology stack, database design, and major modules.
- **Low-Level Design (LLD):** Specifies component logic, APIs, data structures, and workflows.
- **Output:** Design Document Specification (DDS)



➤ Stage 4: Development (Coding)

- Developers build the software based on the approved design.
- **Activities:** Coding, code reviews, unit testing, version control management
- **Tools:** IDEs, version control systems, debuggers
- **Output:** Source code, executable application
- **Key Roles:** Frontend, Backend, Full Stack Developers

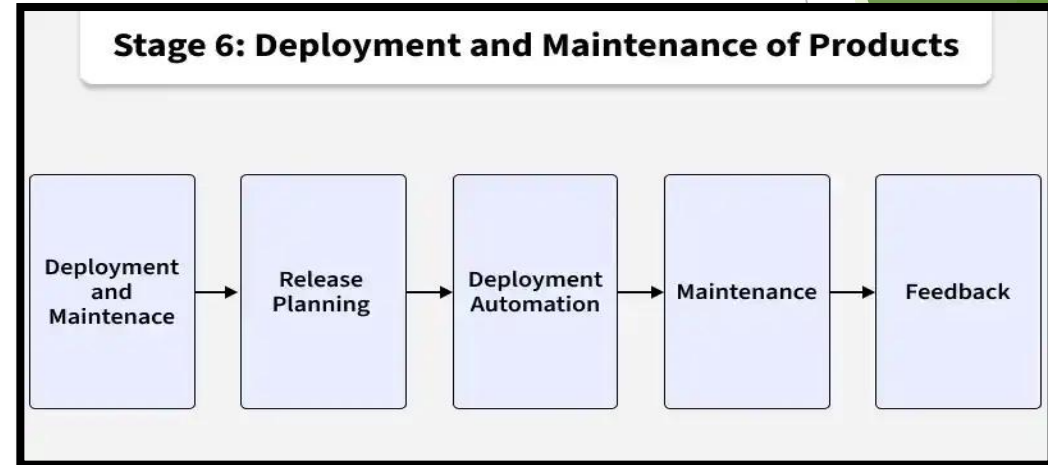


➤ Stage 5: Testing

- Testing ensures the software meets requirements and is free from defects before release.
- **Types of Testing include:**
 - **Unit Testing:** Verifies individual components
 - **Integration Testing:** Ensures modules work together
 - **System Testing:** Validates the complete system
 - **User Acceptance Testing (UAT):** Confirms business requirements are met
- **Output:** Test cases, defect reports, and quality metrics.



- ▶ **Stage 6: Deployment**
- ▶ The tested software is released to users.
- ▶ **Activities:** Production setup, deployment, smoke testing
- ▶ **Modern Approach:** Continuous Integration and Continuous Deployment (CI/CD) pipelines for faster and reliable releases
- ▶ **Output:** Live application
- ▶ **Key Roles:** DevOps Engineers, Release Managers



- ▶ **Stage 7: Maintenance**
- ▶ Post-deployment support ensures long-term usability.
- ▶ **Activities:** Bug fixes, performance tuning, updates, feature enhancements
- ▶ **Output:** Patches, updates, new versions
- ▶ **Key Roles:** Support Engineers, Developers

Software Development Life Cycle Models

- Software Development Models are structured frameworks that guide the planning, execution, and delivery of software projects.
- They define the sequence of development stages, such as requirements, design, coding, testing, and deployment.

1. Waterfall Model

- ▶ Sequential approach
- ▶ Each phase must finish before the next starts
- ▶ Best for small and well-defined projects

2. Agile Model

- ▶ Iterative and flexible
- ▶ Continuous feedback and frequent releases
- ▶ Popular in modern software companies

3. Spiral Model

- ▶ Combines iterative development with risk analysis
- ▶ Suitable for large, high-risk projects

4. V-Model

- Testing is planned alongside development
- Strong focus on quality assurance

5. Iterative Model

- Software is developed in smaller cycles
- Improvements are added repeatedly

6. DevOps Model

- Integrates development and operations
- Emphasizes automation, CI/CD, and continuous delivery

SDLC MODELS

Model	Flexibility to Changes	Risk Management	Cost of Later Changes	Delivery Timeline
Waterfall	Very Low	Low	Very High	Long / At the end
V-Model	Low	Medium	High	Long / At the end
Iterative	Medium	Medium	Medium	Early / Incremental
Spiral	High	Very High	Medium	Incremental
Agile	Very High	High	Low	Very Fast / Continuous
DevOps	Very High	High	Low	Continuous Deployment

Framework	Core Focus	Key Metric
SDLC	What structural steps are needed to build software.	Completeness of architecture and code quality.
Agile	How to build and adapt that software quickly.	Velocity and adaptability to changing requirements.
ITIL	How to run and support that software at scale.	Service availability, uptime, and user satisfaction.

Uptime : The amount of time a system, application, server, or service is operational and available to users without interruption.

Downtime : The period during which a system, application, server, or service is unavailable or not functioning properly.

Agile Software Development

- ▶ Agile Software Development is a software development methodology that values flexibility, collaboration, and customer satisfaction.
- ▶ It is an iterative and incremental approach to the importance of delivering a working product quickly and frequently.
- ▶ It involves close collaboration between the development team and the customer to ensure that the product meets their needs and expectations.
- ▶ **Agile as a Mindset:** Agile represents a shift in culture that values adaptability, collaboration, and client happiness, and it gives team members more authority.
- ▶ **Quick Response to Change:** Agile fosters a culture that allows teams to respond swiftly to constantly shifting priorities and requirements.
- ▶ **Regular Demonstrations:** Agile techniques place a strong emphasis on regular demonstrations of project progress.
- ▶ **Cross-Functional Teams:** Agile fosters self-organising, cross-functional teams that share information effectively and communicate more effectively.

Agile Software Development Process

- ▶ Agile software development, often just called 'Agile', focuses on being flexible and practical when delivering software.
- ▶ Instead of launching everything at once, Agile delivers small, valuable updates to users over time

1. Requirements Gathering

2. Planning

3. Development

4. Testing

5. Deployment

6. Maintenance



1. Requirements Gathering

- ▶ The development team works closely with the customer to understand what they really need from the software.
- ▶ The team listens carefully to the customer's needs, then sorts and priorities these requirements to make sure the most important features are developed first.

2. Planning

- ▶ The team creates a clear plan for how they'll build the software.
- ▶ They decide which features to focus on in each development cycle (called an iteration).

3. Development

- ▶ This is where the team starts turning their plan into reality.
- ▶ They work in short, focused cycles, building small, usable pieces of the product.
- ▶ Each cycle builds on the last, which helps the team stay on track and get quick feedback to keep improving.

4. Testing

- ▶ As the software gets built, it's also tested to make sure it works properly and meets the customer's needs.
- ▶ Testing ensures the product is of high quality and free from errors, so problems are caught early on before they become bigger issues.

5. Deployment

- ▶ Once everything is tested and working as expected, the software is deployed, which means it's ready for customers or end-users to start using.
- ▶ It's the moment when all the development work comes to life.

6. Maintenance

- ▶ Even after the software is released, the job isn't done.
- ▶ The team keeps maintaining the software, verifying it continues to work well and stays up-to-date with any new needs or changes from the customer.

Aspect	Agile	DevOps
Definition	A software development methodology focused on iterative development and customer feedback.	A culture and set of practices that integrate development and operations for continuous delivery.
Objective	Faster development of software through incremental releases.	Faster and reliable software delivery through automation and collaboration.
Focus	Development process and project management.	End-to-end software lifecycle, including development, testing, deployment, and operations.
Team Involvement	Mainly developers, testers, and product owners.	Developers, testers, operations, and system administrators.
Approach	Iterative and incremental development using sprints.	Continuous Integration (CI) and Continuous Delivery/Deployment (CD).

Aspect	Agile	DevOps
Delivery Frequency	Frequent releases after each sprint.	Continuous and automated releases whenever changes are ready.
Automation	Limited focus on automation.	Strong emphasis on automation of build, test, deployment, and monitoring.
Feedback	Customer feedback after each iteration.	Continuous feedback from users, systems, and monitoring tools.
Tools	JIRA, Trello, VersionOne.	JIRA, Jenkins, Docker, Kubernetes, Git, Ansible.
Outcome	Better product development and customer satisfaction.	Faster, stable, and reliable software delivery.

Information Technology Infrastructure Library (ITIL)

- ▶ **Information Technology Infrastructure Library (ITIL)** is a framework of best practices used for managing and delivering IT services effectively.
- ▶ It helps organizations improve service quality, reduce risks, strengthen customer relationships, and ensure cost-effective IT service management.
- ▶ The main goal of ITIL is to **improve efficiency** and provide reliable and predictable IT service delivery.

Basic Terminologies of ITIL

- ▶ **Service:** Delivering value to customers without requiring them to manage specific costs and risks.
- ▶ **Service Management:** A set of specialized capabilities for delivering value through services.
- ▶ **Service Assets:** Resources and capabilities required by a service provider to offer services.
- ▶ **Process:** A structured set of activities designed to achieve a specific objective. A process:
 - ▶ Transforms inputs into outputs.
 - ▶ Delivers results to customers or stakeholders.
 - ▶ Is measurable.
 - ▶ Is triggered by specific events.

Features of ITIL

- ▶ Best-practice framework for IT service management.
- ▶ Provides comprehensive and openly available IT guidance.
- ▶ Defines standards and quality practices for IT services.
- ▶ Has standardized terminology and processes.
- ▶ Developed by the UK's Office of Government Commerce (OGC) in the 1980s.
- ▶ Focuses on improving IT service management and service delivery.

Benefits of ITIL

- ▶ Helps organizations obtain maximum value from IT services.
- ▶ Provides a common language and framework for IT teams.
- ▶ Improves collaboration and service quality.
- ▶ Recognized globally as a standard for IT Service Management (ITSM).
- ▶ Widely used by IT and project management professionals.
- ▶ Supports continuous improvement and efficient service delivery.

Need for DevOps:

- ▶ Organizations today face increasing pressure to deliver high-quality software quickly while maintaining reliability and security.
- ▶ Traditional software development methods often create a gap between development and operations teams, leading to delays, communication issues, and deployment failures.
- ▶ DevOps addresses these challenges by promoting collaboration, automation, and continuous improvement.

1. Faster Software Delivery
2. Improved Collaboration
3. Higher Software Quality
4. Reduced Deployment Failures
5. Faster Problem Resolution
6. Increased Efficiency through Automation
7. Better Scalability and Reliability
8. Enhanced Customer Satisfaction
9. Cost Optimization
10. Support for Agile Development

1. Faster Software Delivery

- ▶ Customers expect frequent updates and new features.
- ▶ Traditional development processes are often slow and time-consuming.
- ▶ DevOps enables Continuous Integration (CI) and Continuous Delivery (CD), reducing release cycles from months to days or even hours.

2. Improved Collaboration

- ▶ Development and operations teams traditionally work in separate silos.
- ▶ Lack of communication leads to misunderstandings and project delays.
- ▶ DevOps encourages shared responsibilities and teamwork, resulting in better coordination and productivity.

3. Higher Software Quality

- ▶ Automated testing identifies bugs early in the development process.
- ▶ Continuous monitoring helps detect and resolve issues quickly.
- ▶ Frequent feedback improves software reliability and performance.

4. Reduced Deployment Failures

- ▶ Manual deployment processes are prone to errors.
- ▶ DevOps uses automation tools to standardize deployments.
- ▶ Consistent deployment practices reduce the risk of system failures.

5. Faster Problem Resolution

- ▶ Continuous monitoring provides real-time insights into system performance.
- ▶ Teams can quickly identify and fix issues.
- ▶ Reduced Mean Time to Recovery (MTTR) improves system availability.

6. Increased Efficiency through Automation

- ▶ Repetitive tasks such as testing, deployment, and infrastructure configuration are automated.
- ▶ Automation reduces human effort and errors.
- ▶ Teams can focus on innovation and value-added activities.

7. Better Scalability and Reliability

- ▶ Infrastructure as Code (IaC) enables automated provisioning and management of resources.
- ▶ Systems can scale efficiently to meet changing business demands.
- ▶ Reliable infrastructure improves application performance.

8. Enhanced Customer Satisfaction

- ▶ Faster delivery of new features improves customer experience.
- ▶ Reliable applications increase user trust.
- ▶ Quick response to customer feedback helps meet user expectations.

9. Cost Optimization

- ▶ Automation reduces operational overhead.
- ▶ Early bug detection minimizes maintenance costs.
- ▶ Efficient resource utilization lowers infrastructure expenses.

10. Support for Agile Development

- ▶ DevOps complements Agile methodologies by enabling continuous development and deployment.
- ▶ It helps organizations adapt quickly to changing requirements and market conditions.

History of DevOps

- DevOps emerged in the late 2000s.
- Created to bridge the gap between Development and Operations teams.
- Addressed communication and efficiency issues in traditional software development.
- Agile methodology (2001) highlighted the need for faster and more efficient deployment processes.
- The term **DevOps** was popularized by Patrick Debois in 2009.
- The first **DevOpsDays** conference was held in 2009.
- Encourages collaboration between developers and operations teams.
- Focuses on automation, continuous integration (CI), and continuous delivery (CD).
- Adoption increased with technologies such as Cloud Computing, Docker, and Kubernetes.
- Enables faster, more reliable, and efficient software delivery.

Evolution of DevOps

Phase / Period	Key Features	Challenges / Outcomes
Traditional Software Development (Before 2000)	• Waterfall model followed sequential development process. • Development, testing, and operations teams worked separately. • Changes were difficult after development started. • Software releases took months or years.	• Poor communication between teams. • Slow release cycles. • High project failure risk. • Delayed feedback and bug fixing.
Agile Movement (2001)	• Introduction of Agile Manifesto. • Customer collaboration. • Continuous feedback. • Iterative development. • Rapid software delivery.	• Operations teams remained separate. • Deployment bottlenecks still existed.

Evolution of DevOps

Phase / Period	Key Features	Challenges / Outcomes
Emergence of DevOps (2007-2009)	• DevOps introduced to bridge the gap between Development and Operations. • Patrick Debois recognized as the "Father of DevOps". • First DevOpsDays conference held in 2009.	• Improved collaboration, automation, and continuous delivery practices.
Growth of DevOps (2010-2015)	• Rise of Cloud Computing. • Increased automation. • Adoption of CI/CD practices. • Tools: Git, Jenkins, Puppet, Chef, Docker.	• Faster releases. • Improved reliability. • Reduced deployment failures.
Modern DevOps Era (2015-Present)	• Cloud Computing. • Containerization (Docker). • Container Orchestration (Kubernetes). • Infrastructure as Code (IaC). • DevSecOps.	• Faster, scalable, secure, and automated software delivery.

Timeline of DevOps

Year	Event
Before 2000	Traditional Waterfall development
2001	Agile Manifesto introduced
2007	Early DevOps concepts emerged
2009	Term "DevOps" popularized and first DevOpsDays conference held
2010-2015	Widespread adoption of CI/CD and automation tools
2013	Docker introduced containerization revolution
2014-2015	Kubernetes introduced container orchestration
2015-Present	DevOps becomes mainstream; DevSecOps emerges

77% of companies globally use DevOps to deploy and manage their software systems

Netflix

Facebook

Microsoft

Meta

Linkedin

Uber

Role of DevOps Engineer

- ▶ A **DevOps Engineer** is responsible for bridging the gap between software development and IT operations teams.
- ▶ They automate processes, improve collaboration, and ensure faster, reliable, and secure software delivery throughout the Software Development Life Cycle (SDLC).

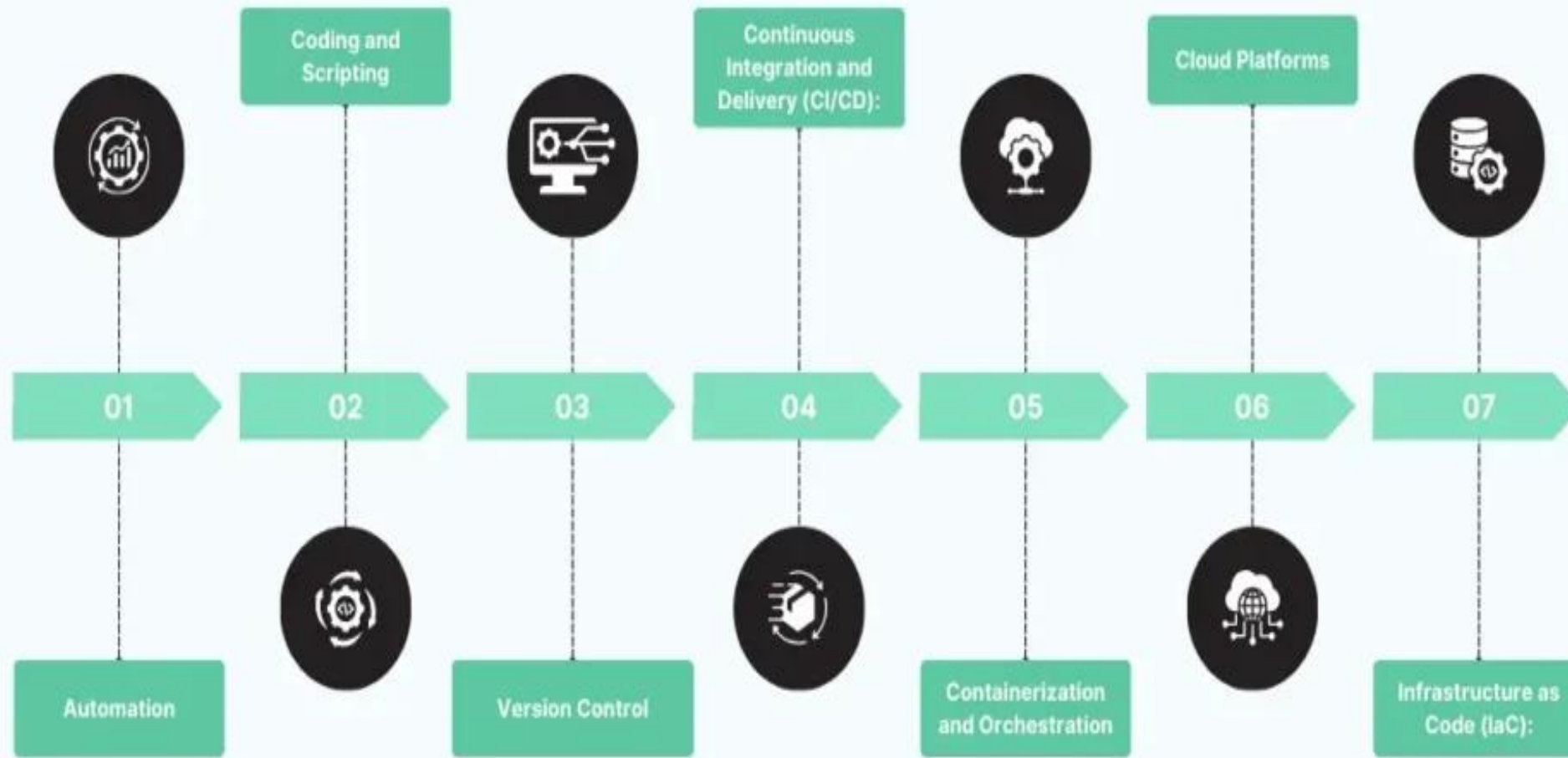
Key Responsibilities

- ▶ Develop and maintain CI/CD pipelines.
- ▶ Automate software testing and deployment.
- ▶ Monitor application and infrastructure health.
- ▶ Manage cloud platforms and containers.
- ▶ Implement security and compliance measures.
- ▶ Ensure high system availability and performance.
- ▶ Support rapid and reliable software releases.

Sr. No.	Role of DevOps Engineer	Description
1	Automation	Automates repetitive tasks such as code integration, testing, deployment, and server configuration to improve efficiency and reduce human errors.
2	CI/CD Pipeline Management	Designs and maintains Continuous Integration (CI) and Continuous Delivery/Deployment (CD) pipelines for faster and smoother software releases.
3	Collaboration	Acts as a bridge between development and operations teams, improving communication and teamwork.
4	Infrastructure Management	Manages cloud and on-premise infrastructure, including servers, databases, and networking resources.
5	Infrastructure as Code (IaC)	Uses tools like Terraform and Ansible to automate infrastructure provisioning and configuration.

Sr. No.	Role of DevOps Engineer	Description
6	Monitoring and Performance Optimization	Monitors applications and systems, identifies performance issues, and ensures high availability and reliability.
7	Security Integration (DevSecOps)	Integrates security practices into the development lifecycle and ensures compliance with security standards.
8	Troubleshooting and Incident Management	Quickly identifies and resolves system failures, deployment issues, and performance bottlenecks.
9	Scalability and Reliability	Ensures systems can handle increasing workloads while maintaining stable performance.
10	Continuous Improvement	Analyzes workflows, removes bottlenecks, and continuously improves software delivery processes.

Technical Skills for a DevOps Engineer



DevOps vs Traditional IT Operations

Aspect	DevOps	Traditional IT Operations
Approach	Collaborative and integrated; combines development and operations teams to work together throughout the lifecycle.	Segmented; development and operations teams work separately, with limited collaboration.
Speed	Fast, with frequent updates and continuous delivery through automation.	Slower, with scheduled releases and manual processes that add delays.
Team Structure	Cross-functional teams that handle development, testing, and deployment together.	Separate teams for development, testing, and deployment, creating silos.
Deployment Frequency	High, with multiple deployments per day possible.	Low, with fewer, scheduled deployments (weekly, monthly, etc.).
Risk Management	Continuous testing and monitoring reduce risk over time.	Risks are managed at release stages, often leading to larger changes and higher impact if issues arise.
Automation	Extensive automation in testing, integration, and deployment processes.	Limited automation, often relying on manual processes.
Feedback Loop	Quick, with immediate feedback loops and fast resolution of issues.	Slower, with feedback gathered at later stages, leading to delays in fixes.
Goal	Speed, reliability, and adaptability to change.	Stability and adherence to established release schedules.

DevOps Basic Terminologies

Term	Description
DevOps	A culture and set of practices that integrate Development (Dev) and Operations (Ops) teams to improve collaboration and software delivery.
CI	Continuous Integration is the practice of automatically merging and testing code changes frequently to detect errors early.
CD	Continuous Delivery or Deployment is the practice of automatically delivering or deploying tested code to production quickly and reliably.
Jenkins	Jenkins is a CI/CD automation tool.
Automation	The use of tools and scripts to perform repetitive tasks such as testing, deployment, and configuration management.
Infrastructure as Code (IaC)	Managing and provisioning infrastructure through code instead of manual configuration.

Term	Description
Version Control	A system that tracks changes in source code and enables collaboration among developers (e.g., Git).
Git	Git is a version control system used to track code changes.
Containerization	Packaging applications and their dependencies into containers for consistent execution across environments (e.g., Docker).
Orchestration	Automated management, deployment, and scaling of containers (e.g., Kubernetes).
Docker	Docker is a containerization platform for packaging and running applications.
Kubernetes	Kubernetes automates the deployment and management of containers.
DevSecOps	Integrating security practices into every stage of the DevOps lifecycle.
Pipeline	An automated workflow that includes building, testing, and deploying software.
Cloud Computing	Delivering computing resources such as servers, storage, and applications over the internet.

Term	Description
SDLC (Software Development Life Cycle)	A structured process used for planning, designing, developing, testing, deploying, and maintaining software applications.
ITIL (Information Technology Infrastructure Library)	A framework of best practices for delivering and managing IT services efficiently and aligning them with business needs.
Agile	A software development methodology that delivers software incrementally through collaboration, continuous feedback, and iterative development.
JIRA	JIRA is an Agile project management and issue-tracking tool.
Scrum	An Agile framework that manages software development through short iterations called sprints, emphasizing teamwork, continuous feedback, and incremental delivery.
Kanban	An Agile workflow management method that uses a visual board to track tasks and improve the continuous flow of work.
Sprint	A fixed-duration iteration (usually 1–4 weeks) in Agile Scrum during which a team completes a set of planned tasks and delivers a working product increment.



JIRA

- **JIRA** is a popular project management and issue-tracking tool developed by [Atlassian](#).
- It is widely used by Agile, Scrum, and DevOps teams to plan, track, manage, and monitor software development projects.

Key Features of JIRA

- ▶ **Issue and Bug Tracking** – Records and manages defects, issues, and enhancement requests.
- ▶ **Project Management** – Helps teams plan, organize, and monitor project activities.
- ▶ **Agile Support** – Supports Agile methodologies such as Scrum and Kanban.
- ▶ **Sprint Planning** – Allows teams to create, manage, and track sprints.
- ▶ **Backlog Management** – Prioritizes and organizes user stories and tasks.
- ▶ **Workflow Management** – Provides customizable workflows for different project needs.
- ▶ **Reporting and Dashboards** – Generates burndown charts, velocity reports, and project status dashboards.
- ▶ **Team Collaboration** – Enables effective communication among developers, testers, project managers, and operations teams.

Sprint in JIRA

- ▶ A **Sprint** is a fixed-duration development cycle, typically lasting **1–4 weeks**, during which a team works on a selected set of tasks or user stories from the backlog.
 - ▶ The goal is to deliver a working product increment by the end of the sprint.
 - ▶ JIRA helps teams plan, execute, and monitor sprints through Scrum boards, sprint backlogs, and progress reports.
-
- ▶ **Scrum** is an Agile project management framework in which work is divided into short, fixed-duration iterations called sprints, allowing teams to deliver working software frequently.

JIRA in DevOps

JIRA is extensively used in **DevOps** to bridge the gap between development and operations teams. It helps in:

- ▶ Tracking development and operational tasks.
- ▶ Managing incidents and service requests.
- ▶ Monitoring software releases and deployments.
- ▶ Coordinating Agile and DevOps workflows.
- ▶ Integrating with CI/CD tools such as GitHub, Bitbucket, and Jenkins.
- ▶ Improving collaboration and visibility across teams.

Uses of JIRA

- ▶ Bug and issue tracking.
- ▶ Project planning and task management.
- ▶ Sprint and backlog management.
- ▶ Monitoring project progress.
- ▶ Supporting Agile and DevOps practices.

Benefits of JIRA

- ▶ Improves team collaboration.
- ▶ Increases project visibility and transparency.
- ▶ Helps track project status in real time.
- ▶ Enhances productivity and workflow management.
- ▶ Supports efficient software development and delivery.